👩‍🏫 RAMÓN ABRAMO • GUÍA TÉCNICA DETALLADA

Versión 3: Arquitectura POO & Audio

Diseño de software empresarial con clases ES6, Web Audio API nativo, confeti en Canvas 2D y persistencia con localStorage.

La **Versión 3** traslada el ejercicio al nivel de un producto comercial terminado. Aplica el paradigma de Programación Orientada a Objetos (POO) para separar drásticamente la lógica pura del juego (Engine) de la manipulación de la interfaz (UI), el audio sintético y el almacenamiento local persistente.

Estructura de la Carpeta `version-3-avanzada-poo/`

Archivo	Propósito	Destinatario
`index.html`	Selector de dificultad, temporizador, barra de salud, modales y canvas.	Alumno / Docente
`main.css`	Diseño gaming premium, barra de salud dinámica, animaciones y modales.	Alumno / Docente
`app.js`	Código maestro con las 4 clases completas y comentarios formativos.	Docente
`app.alumnos.js`	Plantilla con la arquitectura de clases y bloques `TODO`.	Alumnado
`app.solucion.js`	Solución limpia de referencia para el profesor.	Docente
`presentacion.html`	Diapositivas interactivas para proyectar en clase.	Docente en Aula

Arquitectura Modular de Clases ES6

1. AhorcadoEngine

Motor lógico puro. Maneja el estado, las reglas, las vidas y los 3 niveles de dificultad sin tocar el DOM.

2. SoundSynthesizer

Genera ondas sonoras con Web Audio API puro. No depende de archivos mp3 externos ni conexiones de red.

3. StatsRepository

Almacena estadísticas permanentes en `localStorage`: partidas, victorias, derrotas y rachas récord.

4. AhorcadoUI & Confetti

Coordina eventos de usuario, modales interactivos y renderiza confeti físico a 60 FPS sobre un `<canvas>`.

Módulo de Audio Nativo: `SoundSynthesizer`

> En videojuegos web para el aula, depender de archivos de audio externos (`.mp3`) genera problemas de rutas relativas o bloqueos CORS locales. La solución profesional es sintetizar el sonido con la Web Audio API estándar:

```
class SoundSynthesizer {
  constructor() {
    this.audioCtx = null;
    this.muted = localStorage.getItem('ahorcado_muted') === 'true';
  }

  initContext() {
    if (!this.audioCtx) {
      const AudioCtx = window.AudioContext || window.webkitAudioContext;
      if (AudioCtx) this.audioCtx = new AudioCtx();
    }
  }

  playTone(frecuencia, tipo, duracion, retardo = 0) {
    if (this.muted) return;
    this.initContext();
    if (!this.audioCtx) return;

    const osc = this.audioCtx.createOscillator();
    const gain = this.audioCtx.createGain();

    osc.type = tipo; // 'sine', 'triangle', 'sawtooth'
    osc.frequency.setValueAtTime(frecuencia, this.audioCtx.currentTime + retardo);

    // Atenuación suave para evitar chasquidos en los altavoces
    gain.gain.setValueAtTime(0.12, this.audioCtx.currentTime + retardo);
    gain.gain.exponentialRampToValueAtTime(0.001, this.audioCtx.currentTime + retardo + duracion);

    osc.connect(gain);
    gain.connect(this.audioCtx.destination);
    osc.start(this.audioCtx.currentTime + retardo);
    osc.stop(this.audioCtx.currentTime + retardo + duracion);
  }

  playSuccess() { this.playTone(587.33, 'sine', 0.15); } // Re5 agudo
  playError()   { this.playTone(180.00, 'sawtooth', 0.2); } // Tono grave
  playVictory() {
    [523, 659, 783, 1046].forEach((f, i) => this.playTone(f, 'triangle', 0.2, i * 0.08));
  }
}
```

Autoplay Policy del Navegador

Los navegadores modernos no permiten emitir sonido hasta que el usuario interactúa (hace clic). Por eso `initContext()` se ejecuta perezosamente (*lazy initialization*) en la primera jugada.

Módulo de Persistencia: `StatsRepository`

Conserva el progreso del jugador entre sesiones (partidas, victorias, rachas y récords) sin necesidad de backend o servidor de base de datos:

```
class StatsRepository {
  constructor() {
    this.KEY = 'ahorcado_stats_pro_2026';
    this.stats = this.load();
  }

  load() {
    try {
      const data = localStorage.getItem(this.KEY);
      if (data) return JSON.parse(data);
    } catch (e) {}
    return { partidas: 0, victorias: 0, derrotas: 0, rachaActual: 0, mejorRacha: 0 };
  }

  save() {
    localStorage.setItem(this.KEY, JSON.stringify(this.stats));
  }

  recordGame(victoria) {
    this.stats.partidas++;
    if (victoria) {
      this.stats.victorias++;
      this.stats.rachaActual++;
      if (this.stats.rachaActual > this.stats.mejorRacha) {
        this.stats.mejorRacha = this.stats.rachaActual;
      }
    } else {
      this.stats.derrotas++;
      this.stats.rachaActual = 0;
    }
    this.save();
  }

  get winPercentage() {
    return this.stats.partidas ? Math.round((this.stats.victorias / this.stats.partidas) * 100) : 0;
  }
}
```

Getters Computados en Clases ES6

El método `get winPercentage()` permite acceder al porcentaje como si fuera una propiedad regular (`repo.winPercentage`), calculándose dinámicamente cada vez que se consulta.

El Motor Lógico: `AhorcadoEngine`

> El motor no contiene una sola referencia al DOM (ni `document`, ni `window`). Es código puro y testeable que gestiona las reglas y los niveles de dificultad:

```
class AhorcadoEngine {
  constructor() {
    this.dificultad = 'medio';
    this.configs = {
      facil:   { maxFallos: 8, tiempoMax: null, iniciales: 2 },
      medio:   { maxFallos: 6, tiempoMax: null, iniciales: 0 },
      dificil: { maxFallos: 5, tiempoMax: 45,   iniciales: 0 }
    };
  }

  nuevaPartida() {
    this.item = BANCO_DATOS[Math.floor(Math.random() * BANCO_DATOS.length)];
    this.adivinadas = Array(this.item.palabra.length).fill('');
    this.usadas = new Set();
    this.fallos = 0;
    this.terminado = false;

    const conf = this.getConfig();
    if (conf.iniciales > 0) {
      // Revelar 2 letras iniciales en modo fácil
      const unicas = [...new Set(this.item.palabra.split(''))];
      for (let i = 0; i < conf.iniciales && i < unicas.length; i++) {
        const l = unicas[i];
        this.usadas.add(l);
        for (let j = 0; j < this.item.palabra.length; j++)
          if (this.item.palabra[j] === l) this.adivinadas[j] = l;
      }
    }
    return { ...this.item, maxFallos: conf.maxFallos, tiempoMax: conf.tiempoMax };
  }

  probarLetra(letra) {
    if (this.terminado || this.usadas.has(letra)) return null;
    this.usadas.add(letra);

    const acierto = this.item.palabra.includes(letra);
    if (acierto) {
      for (let i = 0; i < this.item.palabra.length; i++)
        if (this.item.palabra[i] === letra) this.adivinadas[i] = letra;
    } else {
      this.fallos++;
    }

    const victoria = !this.adivinadas.includes('');
    const derrota = this.fallos >= this.getConfig().maxFallos;
    if (victoria || derrota) this.terminado = true;

    return { letra, acierto, fallos: this.fallos, victoria, derrota, palabra: this.item.palabra };
  }
}
```

El Controlador: `AhorcadoUI` y Confeti Canvas

`AhorcadoUI` actúa como orquestador central: recibe los clics y pulsaciones, consulta a `AhorcadoEngine`, activa sonidos con `SoundSynthesizer` y lanza el confeti:

```
class AhorcadoUI {
  jugar(letra) {
    const res = this.engine.probarLetra(letra);
    if (!res) return;

    const btn = this.dom.teclado.querySelector(`[data-letra="${letra}"]`);
    if (btn) { btn.disabled = true; btn.classList.add(res.acierto ? 'acierto' : 'fallo'); }

    if (res.acierto) {
      this.audio.playSuccess();
      this.renderPalabra();
    } else {
      this.audio.playError();
      if (this.dom.partesSvg[res.fallos - 1]) {
        this.dom.partesSvg[res.fallos - 1].classList.add('visible');
      }
      this.actualizarSalud();
    }

    if (res.victoria || res.derrota) this.finPartida(res);
  }

  finPartida(res) {
    clearInterval(this.timer);
    this.stats.recordGame(res.victoria);

    if (res.victoria) {
      this.audio.playVictory();
      this.confetti.start(); // Partículas físicas en <canvas>
      this.dom.modalTitulo.textContent = '¡Victoria Brillante!';
    } else {
      this.audio.playDefeat();
      this.dom.caraDerrota.classList.add('visible');
      this.dom.modalTitulo.textContent = 'Fin de la Partida';
    }
    this.dom.modalFin.classList.remove('oculta');
  }
}
```

Física de Partículas en Canvas

La clase `ConfettiAnimation` genera 120 partículas con gravedad (`p.y += p.speedY`), oscilación horizontal (`p.x += Math.sin( ... )`) y rotación a 60 FPS con `requestAnimationFrame()` .

Resumen Didáctico de la Versión 3

> Esta solución demuestra a los alumnos cómo escalar un desarrollo web sin crear "código espagueti". Cada clase tiene un único propósito bien delimitado.

Patrón Modelo-Vista (MVC básico)

- **Modelo:** `AhorcadoEngine` y `StatsRepository`.
- **Vista:** El DOM, SVG y el lienzo `<canvas>`.
- **Controlador:** `AhorcadoUI` que escucha los eventos y coordina las llamadas.

Por qué es un proyecto de Portafolio

Cualquier reclutador o empresa que evalúe este código verá: dominio de POO, arquitectura de eventos, APIs modernas de navegador (Audio, Canvas, Storage) y experiencia de usuario profesional.

Rúbrica de Corrección para la Versión 3

Aspecto	Excelente (10)	Mejorable (< 7)
Desacoplamiento	Engine no tiene llamadas al DOM.	Engine manipula elementos HTML directamente.
Web Audio API	Sonidos limpios con rampas de ganancia.	Uso de archivos externos que fallan en local.
LocalStorage	Serialización limpia con bloque `try/catch`.	Errores si el navegador tiene cookies bloqueadas.

¡Enhorabuena! Has completado el análisis de la Versión 3 Avanzada POO. 🚀 🎓